



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251230
Nama Lengkap	Okky Alexander
Minggu ke / Materi	13 / Tipe Data Tuples

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

MATERI TUPLE IMMUTABLE

Tuple mirip dengan list, namun memiliki tiga perbedaan utama: bersifat immutable (elemennya tidak dapat diubah), comparable (dapat dibandingkan), dan hashable (dapat digunakan sebagai key dalam dictionary).

Objek hashable adalah objek yang nilai hash-nya tidak pernah berubah selama masa hidupnya melalui method `__hash__()` dan dapat dibandingkan menggunakan `__eq__()`. Secara umum, objek immutable bawaan Python bersifat hashable, sedangkan objek mutable seperti list dan dictionary tidak.

Tuple dapat ditulis dengan beberapa cara, yaitu tanpa tanda kurung, dengan tanda kurung, maupun menggunakan fungsi bawaan `tuple()`.

```
>>> data = 1, 2, 3, 4, 5    # tanpa kurung atau >>> data = (1, 2, 3, 4, 5)    # dengan kurung
```

Untuk tuple dengan satu elemen, tanda koma di belakang elemen wajib ditambahkan. Tanpanya, Python akan menganggapnya sebagai tipe data biasa, bukan tuple.

```
>>> data1 = (10,)    # ini tuple /// >>> data2 = (10)    # ini integer
```

Fungsi `tuple()` tanpa argumen menghasilkan tuple kosong, sedangkan jika diberi argumen berupa urutan seperti string atau list, fungsi ini akan mengubahnya menjadi tuple.

```
>>> tuple()
```

```
>>> tuple('kampus') output : ('k', 'a', 'm', 'p', 'u', 's')
```

Karena tuple adalah nama constructor bawaan Python, kata tersebut tidak boleh digunakan sebagai nama variabel. Sebagian besar operator yang berlaku pada list juga berlaku pada tuple, termasuk pengaksesan elemen menggunakan indeks `[]` dan *slicing*.

```
>>> data = (10, 20, 30, 40, 50) /// >>> print(data[0])    # akses elemen /// >>> print(data[1:3])    # slicing
```

Karena tuple bersifat immutable, mengubah elemennya secara langsung akan menghasilkan error. Namun tuple tetap bisa diperbarui dengan cara menggabungkan elemen baru dengan irisan tuple yang sudah ada.

```
>>> data[0] = 99
```

```
TypeError: object doesn't support item assignment
```

```
>>> data = (99,) + data[1:]
```

MATERI MEMBANDINGKAN TUPLE

Operator perbandingan pada tuple bekerja dengan membandingkan elemen pertama terlebih dahulu, lalu berlanjut ke elemen berikutnya jika ditemukan kesamaan.

```
>>> (0, 1, 5) < (0, 3, 2) // True
```

```
>>> (0, 1, 9999) < (0, 3, 2) // True
```

Fungsi sort bekerja dengan prinsip yang sama. Konsep ini dikenal sebagai DSU (Decorate, Sort, Undecorate) yang terdiri dari tiga tahap:

1. Decorate — membangun daftar tuple dengan menyertakan kunci pengurutan sebelum elemen aslinya.
2. Sort — mengurutkan daftar tuple menggunakan fungsi sort.
3. Undecorate — mengekstrak kembali elemen yang telah terurut menjadi satu sekuensial.

```
kalimat = 'to be or not to be that is the question'
daftar_kata = kalimat.split()
t = list()
for kata in daftar_kata:
    t.append((len(kata), kata))

t.sort(reverse=True)

urutan = list()
for panjang, kata in t:
    urutan.append(kata)

print(urutan)
```

Gambar 13.1 : Code Program.

Perulangan pertama membangun daftar tuple berisi setiap kata beserta panjangnya. Fungsi sort kemudian mengurutkan berdasarkan panjang kata, dan apabila panjangnya sama, pengurutan dilanjutkan secara alfabetis. Parameter `reverse=True` membuat hasil urutan menjadi menurun (*descending*). Perulangan kedua lalu menyusun kembali kata-kata sesuai urutan yang telah dihasilkan, sehingga keluarannya adalah sebagai berikut.

```
['question', 'that', 'not', 'be', 'or', 'to', 'to', 'be', 'is', 'the']
```

Kata-kata tersebut tersusun dari yang memiliki jumlah karakter terbanyak hingga paling sedikit.

MATERI PENUGASAN TUPLE

Salah satu fitur unik Python adalah kemampuan menempatkan tuple di sisi kiri pernyataan penugasan, sehingga lebih dari satu variabel dapat ditetapkan sekaligus dalam satu baris.

```
>>> data = ['belajar', 'python']
```

```
>>> p, q = data
```

```
>>> p // 'belajar'
```

```
>>> q // 'python'
```

Secara internal, Python menerjemahkannya menjadi penugasan satu per satu ($p = data[0]$, $q = data[1]$).

Penulisan dengan tanda kurung juga menghasilkan cara kerja yang sama ($(p, q) = data$).

Fitur ini juga memungkinkan pertukaran nilai antar variabel tanpa membutuhkan variabel sementara.

```
>>> x, y = y, x
```

Yang perlu diperhatikan, jumlah variabel di sisi kiri harus sama dengan jumlah nilai di sisi kanan, jika tidak Python akan memunculkan error.

```
>>> x, y = 1, 2, 3
```

ValueError: too many values to unpack

Fitur ini juga sering dimanfaatkan untuk memecah data sekuensial, misalnya memisahkan alamat email menjadi nama pengguna dan domain menggunakan `split()`.

```
>>> email = 'budiono@kampus.ac.id'
```

```
>>> namapengguna, domain = email.split('@')
```

```
>>> print(namapengguna) /// output : budiono
```

```
>>> print(domain) // output : kampus.ac.id
```

MATERI DICTIONARIES AND TUPLE

Metode `items()` pada dictionary berfungsi mengembalikan list of tuple, di mana setiap tuple berisi satu pasangan kunci dan nilai (*key-value pair*).

```
>>> nilai = {'x': 5, 'y': 30, 'z': 17}
```

```
>>> t = list(nilai.items())
```

```
>>> print(t) // output : [('y', 30), ('x', 5), ('z', 17)]
```

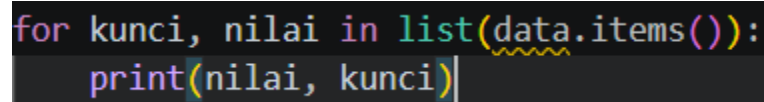
Karena dictionary tidak memiliki urutan tertentu, hasil yang dikembalikan pun tidak terurut. Namun karena keluarannya berupa list of tuple, kita dapat menggunakan `sort()` untuk mengurutkannya berdasarkan kunci secara *ascending*.

```
>>> t.sort()
```

```
>>> t // output : [('x', 5), ('y', 30), ('z', 17)]
```

MATERI MULTIPENUGASAN DENGAN DICTIONARIES

Kombinasi `items()`, penugasan tuple, dan perulangan `for` memungkinkan akses kunci dan nilai dictionary sekaligus dalam satu perulangan. Setiap iterasi, pasangan kunci-nilai secara otomatis dipisahkan ke masing-masing variabel.



```
for kunci, nilai in list(data.items()):  
    print(nilai, kunci)
```

Gambar 13.2 : Code Program.

Apabila ingin mengurutkan dictionary berdasarkan nilainya, susunan tuple perlu dibalik menjadi (*nilai, kunci*) agar pengurutan dilakukan berdasarkan nilai, bukan kunci.

```
>>> data = {'p': 40, 'q': 7, 'r': 55}
```

```
>>> l = list()
```

```
>>> for kunci, nilai in data.items():
```

```
...     l.append((nilai, kunci))
```

```
>>> l.sort(reverse=True)
```

```
>>> l
```

```
[(55, 'r'), (40, 'p'), (7, 'q')]
```

Dengan menempatkan nilai sebagai elemen pertama, isi dictionary dapat ditampilkan terurut dari nilai terbesar hingga terkecil.

MATERI KATA YANG SERING MUNCUL

Program ini bertujuan menampilkan 10 kata yang paling sering muncul dalam sebuah file teks menggunakan list of tuple.

```
import string
berkas = open('contoh-teks.txt')
frekuensi = dict()
for baris in berkas:
    baris = baris.translate(str.maketrans('', '', string.punctuation))
    baris = baris.lower()
    kata_kata = baris.split()
    for kata in kata_kata:
        if kata not in frekuensi:
            frekuensi[kata] = 1
        else:
            frekuensi[kata] += 1

# urutkan dictionary berdasarkan nilai
daftar = list()
for kunci, nilai in list(frekuensi.items()):
    daftar.append((nilai, kunci))

daftar.sort(reverse=True)

for nilai, kunci in daftar[:10]:
    print(nilai, kunci)
```

Gambar 13.3 : Code Program.

Cara kerjanya: file dibaca baris per baris, tanda baca dihapus menggunakan `translate()`, lalu teks diubah ke huruf kecil agar perhitungan tidak membedakan huruf besar dan kecil. Setiap kata kemudian dihitung kemunculannya dalam dictionary. Selanjutnya, dictionary diubah menjadi list of tuple bersusunan *(frekuensi, kata)* agar pengurutan dilakukan berdasarkan frekuensi. Jika dua kata memiliki frekuensi sama, keduanya diurutkan secara alfabetis. Terakhir, 10 kata teratas dicetak menggunakan `daftar[:10]`.

Kemampuan melakukan analisis data yang cukup kompleks hanya dalam sekitar 19 baris kode menjadi salah satu alasan mengapa Python banyak dipilih untuk keperluan eksplorasi dan analisis informasi.

MATERI TUPLE SEBAGAI KUNCI DICTIONARIES

Karena tuple bersifat hashable, tuple dapat digunakan sebagai kunci dalam dictionary, termasuk sebagai *composite key* yaitu kunci yang terdiri dari lebih dari satu nilai. Contoh penerapannya adalah direktori telepon yang menggunakan kombinasi nama belakang dan nama depan sebagai kunci.

python

```
direktori[nama_belakang, nama_depan] = nomor_telepon
```

Ekspresi di dalam kurung kotak tersebut sejatinya adalah sebuah tuple. Untuk mengakses seluruh isinya, dapat menggunakan perulangan for dengan penugasan tuple sekaligus.

```
for nama_belakang, nama_depan in direktori:  
    print(nama_depan, nama_belakang, direktori[nama_belakang, nama_depan])
```

Gambar 13.4 : Code Program.

```
>>> direktori['pratama', 'bagas'] = '081234567890'
```

```
>>> for nb, nd in direktori:
```

```
...     print(nd, nb, direktori[nb, nd])
```

```
...
```

```
bagas pratama 081234567890
```

Source Github : https://github.com/tuangkeman/71251230_Okky.git

LATIHAN 13.1

```
tA = (90, 90, 90, 90)  
result = len(set(tA)) == 1  
print(result)
```

Gambar 13.5 : Code Program.

Program ini bekerja dengan mengubah tuple tA menjadi sebuah set menggunakan fungsi set(tA). Karena set secara otomatis menghapus nilai-nilai yang duplikat, maka jika semua anggota tuple bernilai sama, set hanya akan berisi tepat 1 elemen unik. Kemudian len(...) == 1 digunakan untuk mengecek apakah jumlah elemen dalam set tersebut adalah 1, dan hasilnya dikembalikan sebagai True atau False.

LATIHAN 13.2

```
d = ('Matahari Bhakti Nendya', '22064091', 'Bantul, DI Yogyakarta')

jeneng = d[0]
nim = d[1]
dodos = d[2]

print("Data: ", d)
print()
print("NIM      :", nim)
print("NAMA     :", jeneng)
print("ALAMAT  :", dodos)
print()

nt = tuple(nim)
print("NIM:", nt)
print()

jenngarep = jeneng.split()[0]
ndt = tuple(jenngarep)
print("NAMA DEPAN:", ndt)
print()

walik = tuple(reversed(jeneng.split()))
print("NAMA TERBALIK:", walik)
```

Gambar 13.5 : Code Program.

Program ini menyimpan data diri berupa nama, NIM, dan alamat ke dalam sebuah tuple bernama data sesuai contoh pada soal. Setiap elemen diakses menggunakan indeks, lalu ditampilkan sebagai informasi NIM, NAMA, dan ALAMAT. Selanjutnya, NIM diubah menjadi tuple karakter menggunakan fungsi tuple(). Nama depan diambil dari kata pertama yaitu 'Matahari' menggunakan split()[0], lalu dipecah menjadi tuple karakter. Terakhir, nama dibalik urutan katanya menggunakan reversed() pada hasil split(), sehingga menghasilkan tuple ('Nendya', 'Bhakti', 'Matahari').

LATIHAN 13.3

```
file = input("Enter a file name: ")

c = dict()

try:
    fop = open(file)
    for line in fop:
        if not line.startswith("From "):
            continue
        kata = line.split()
        waktu = kata[5]
        jam = waktu.split(":")[0]
        c[jam] = c.get(jam, 0) + 1

    for jam in sorted(c):
        print(jam, c[jam])
except FileNotFoundError:
    print("File tidak ditemukan:", file)
```

Gambar 13.6 : Code Program.

Program ini membaca file teks yang berisi data email, lalu menghitung berapa banyak pesan yang diterima di setiap jam dalam sehari. Pertama, program meminta input nama file dari pengguna. Kemudian setiap baris yang diawali dengan "From " diproses karena baris tersebut mengandung informasi pengirim dan waktu pengiriman email. Waktu diambil dari elemen ke-5 pada baris tersebut, lalu jam dipisahkan menggunakan split(":"). Setiap jam dicatat ke dalam dictionary counts dengan cara menambah nilainya setiap kali jam yang sama muncul. Terakhir, hasil ditampilkan secara terurut menggunakan sorted() sehingga jam ditampilkan dari yang terkecil hingga terbesar beserta jumlah pesannya.